

# Diffusion Policy: Is Scaling All You Need?

Jack Benarroch Jedlicki  
Computer Science Department,  
Harvard University  
Cambridge, USA  
jackbena@mit.edu

Kushal Chattopadhyay  
Computer Science Department,  
Harvard University  
Cambridge, USA  
kushalch@mit.edu

**Abstract**—Recent advances in robot learning have shown that diffusion policies achieve state-of-the-art performance in learning manipulation skills from demonstrations. However, generalization remains a crucial capability for physical intelligence: a robot trained in one setup should adapt to changes in object placement, shape, or the presence of obstacles. While humans generalize effortlessly, learned policies often fail under such distribution shifts. In this project, we study the generalization properties of diffusion policies and ask: How well do they perform under unseen environmental variations, and how does performance scale with additional training data? We investigate these questions in a tabletop cube-stacking task, evaluating generalization to novel object positions and to scenarios involving obstacles. Our experiments reveal that diffusion policies generalize strongly when provided even small amounts of out-of-distribution (OOD) data; with enough OOD demonstrations, performance approaches in-distribution (ID) levels without degrading ID performance. These findings highlight the data-driven adaptability of diffusion policies and their potential as scalable components of general-purpose robotic control systems.

**Index Terms**—Diffusion policy, imitation learning, generalization, robot learning, robotic manipulation.

## I. INTRODUCTION

In recent years, diffusion-based models have become increasingly prominent in physical artificial intelligence, largely due to their ability to generate flexible and expressive action distributions through iterative denoising. Although new tasks and environments for evaluating diffusion policies continue to emerge, many widely used benchmarks—such as Robomimic [1] and Franka Kitchen [2]—primarily assess imitation learning or offline reinforcement learning capabilities. In these settings, Diffusion Policy is known to produce multimodal action distributions, often discovering multiple equally effective solutions to a given manipulation task (e.g., Push-T).

While prior research has focused on this multimodality and the ability to uncover diverse high-reward trajectories, our interest lies instead in the *generalization* properties of Diffusion Policy (DP): how well it adapts to new tasks and environments, particularly under distribution shifts such as novel object placements or the introduction of obstacles. In particular, our goal is to characterize how DP generalization scales with structured, incremental out-of-distribution (OOD) data under controlled task variations. To investigate this, we conduct a series of targeted experiments comparing DP performance in both in-distribution (ID) and OOD environments.

We focus on a tabletop manipulation task involving stacking a red cube on top of a green cube. We study two OOD settings: OOD-positions, where the cubes are placed in new regions of the table (Subsection IV-A), and OOD-obstacles, where an obstacle is placed between the two cubes (Subsection IV-B). In each case, we begin by training a baseline model on 200 ID demonstrations and confirm high ID success rates (above 90%). However, when evaluated in the corresponding OOD environment, performance drops markedly. To quantify how OOD generalization scales with additional data, we design motion planners that solve the tasks in the OOD domain, collect new OOD demonstrations, and incrementally augment the training set by injecting increasing amounts of OOD data while keeping all other training conditions fixed. For each dataset variant, we retrain the model from scratch and evaluate how OOD success changes as a function of the number of OOD demonstrations.

Across both settings, we observe a consistent trend: diffusion policies exhibit strong data-driven generalization. Even a small number of OOD demonstrations yields substantial performance improvements, and with enough OOD data, performance approaches ID levels. These results, obtained using controlled environments built in ManiSkill [3], and systematic ablations tracked with Weights & Biases, suggest that diffusion policies can be efficiently adapted to new environments with minimal additional supervision—an encouraging property for scalable and general robotic foundation models.

The code has been made available in the repository [github.com/JackBJ23/scaling-dp](https://github.com/JackBJ23/scaling-dp), and all videos mentioned are in [scaling-dp/videos](#).

## II. RELATED WORK

Prior work has investigated scaling laws and generalizability of robot policies across environments, tasks, and datasets, aiming to quantify how distributional shifts affect policy performance. To contextualize generalization more formally, taxonomies such as STAR-Gen [4] decompose policy performance along three axes—visual, semantic, and behavioral—corresponding respectively to shifts in sensory appearance, object/task semantics, or task structure. Such frameworks have been implemented to show that state-of-the-art vision-language-action (VLA) foundation models exhibit strong in-distribution performance yet suffer 20–60% degradation under modest semantic OOD shifts such as novel object categories,

attribute changes, or altered task constraints [4]. These results highlight that even at scale current robotic foundation models lack reliable generalization without explicit, targeted distributional data coverage.

Diffusion-based visuomotor policies have emerged as a promising alternative due to their stochastic action-sequence modeling and ability to represent multimodal action distributions. The original Diffusion Policy [5] demonstrated substantial improvements over prior imitation-learning baselines across a variety of manipulation tasks, and subsequent follow-up work—including DP3 [6] and post-training augmentation pipelines [7]—has shown that diffusion policies can generalize within-distribution and can often recover near-expert performance using only a small number of additional demonstrations. However, despite these encouraging findings, existing studies rarely offer an explicit characterization of how generalization arises in diffusion-based controllers. Moreover, most ablations evaluate generalization only across entirely new tasks, leaving open questions regarding the policy’s sensitivity to small but systematic perturbations such as shifts in object placement, color, size, or minor environmental obstacles.

Parallel lines of work investigate enhancing diffusion-policy performance through offline RL, reward shaping, and joint imitation–reinforcement approaches [8–10] demonstrating improved robustness in multimodal or visually challenging scenarios. These studies further support the intuition that diffusion models possess powerful representational capacity, but they do not fully examine the incremental effects of structured OOD data on policy adaptation. Motivated by this gap, we conduct a series of controlled ablations and OOD generalization experiments to analyze how diffusion policies respond to targeted, stepwise increases in OOD demonstrations. Our evaluation focuses on variations in object positions and on the introduction of obstacles, enabling a systematic investigation of the scaling behavior of diffusion-policy generalization under small, interpretable distribution shifts.

### III. APPROACH

#### A. Diffusion Policy

Diffusion Policies [5] formulate visuomotor control as a generative modeling problem using Denoising Diffusion Probabilistic Models (DDPMs) [11]. A DDPM generates samples by iteratively denoising a Gaussian noise vector  $\mathbf{A}^K \sim \mathcal{N}(0, I)$  through a sequence  $\mathbf{A}^K \rightarrow \mathbf{A}^{K-1} \rightarrow \dots \rightarrow \mathbf{A}^0$ , where  $\mathbf{A}^0$  is the final action sequence. Each reverse step removes a small amount of noise using a learned predictor  $\varepsilon_\theta$ , with the generic update

$$\mathbf{A}_t^{k-1} = \alpha(\mathbf{A}_t^k - \gamma \varepsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^k, k)) + \mathcal{N}(0, \sigma^2 I).$$

Training corrupts expert demonstrations (from teleoperation or motion-planning trajectories) by adding Gaussian noise  $\varepsilon_k \sim \mathcal{N}(0, \sigma_k^2 I)$  and asks the model to predict this noise using the mean squared error (MSE) loss:

$$\mathcal{L} = \text{MSE}(\varepsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^0 + \varepsilon_k, k), \varepsilon_k).$$

Here,  $\mathbf{A}_t^k$  denotes the noisy future action sequence,  $\mathbf{A}_t^0$  the ground-truth actions, and  $\mathbf{O}_t$  the current observations. At inference time, the policy receives an observation window of length  $T_o$  and predicts a future action sequence of length  $T_p$ , executing only the first  $T_a < T_p$  actions before replanning. Following [5], we use  $T_o=2$ ,  $T_p=16$ , and  $T_a=8$ .

Additionally, to generate the ground-truth actions, we build motion planners in the ManiSkill simulation framework. Before training, each demonstration is segmented into shorter action sequences  $\mathbf{A}_t$  of fixed length  $T_p=16$ , together with their corresponding conditioning observations  $\mathbf{O}_t$ , yielding the final training dataset.

#### B. Environments

An essential component of our pipeline is the construction of the training and evaluation environments. To do so, we use the ManiSkill simulation framework [3], which builds upon the SAPIEN simulator [12, 13]. We focus on a pick-and-place task in which the robot must pick up a red cube from a table, stack it on top of a green cube, and release it without causing either cube to fall. An episode is considered successful if, at termination, the red cube is resting on top of the green cube, the red cube is static, and the robot is no longer grasping it. The robot is a Panda arm mounted on the table, and we operate in state-based observation mode, where the policy receives a vector describing the robot state and the poses of the objects in the scene. Our goal is to study the generalization of diffusion policies to OOD scenarios. In particular, we investigate two types of distribution shift: (1) novel object positions and (2) the presence of obstacles. We construct training and evaluation environments for each case.

*a) OOD positions:* We define region A on the table as  $\{(x, y) \in [5, 25] \times [-15, 15]\}$  and region B as its reflection across the  $y$ -axis, given by  $\{(x, y) \in [-25, -5] \times [-15, 15]\}$  (all distances in cm); see Figure 1. The model is first trained on demonstrations in which both cubes lie in region A, and then evaluated in an OOD setting where both cubes are placed in region B. We refer to this task as *OOD-B*. During evaluation, the  $z$ -axis rotations of both cubes are randomized, and their  $(x, y)$  positions are sampled uniformly within the region such that the cubes do not collide. We additionally introduce a second OOD task, *OOD-C*, in which both cubes are placed in region C, defined as  $\{(x, y) \in [-4, 4] \times [-15, 15]\}$ , i.e., the area between regions A and B. Since region C is unseen during training—even when training with additional demonstrations from region B—this setup allows us to explore whether the policy exhibits an *interpolation effect*, where combining data from regions A and B enhances generalization to the space between them.

*b) OOD obstacle:* In this case, the red cube is placed in  $\{(x, y) \in [-15, 15] \times [-20, -10]\}$  and the green cube in  $\{(x, y) \in [-15, 15] \times [-5, 5]\}$ , positioning the red cube closer to the robot base and the green cube farther away. We introduce an obstacle by placing a blue block approximately midway along the path between the two cubes, with an added random offset  $(\tilde{x}, \tilde{y}) \in (-1, 1) \times (-1, 1)$ . These region choices

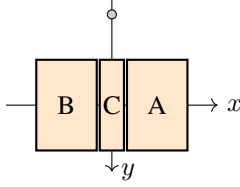


Fig. 1. Regions A, B, and C for the OOD position experiments; the grey disk represents the base of the Panda arm.

ensure that the resulting trajectories can be executed reliably in all scenarios. In our experiments, alternative placements—for instance, positioning the red cube farther from the robot than the green cube—often produced undesirable collisions between the block and the robot arm. We also assign the obstacle block a very high density so that, even if the robot attempts to move directly toward the green cube to complete the stack, it will collide with the block rather than pushing it aside, preventing task completion through a trivial straight trajectory. Finally, we note that the blue block is always present in the scene in order to maintain a fixed observation dimensionality for state-based policies. Thus, in the “no-obstacle” condition, we place the block at  $(x, y) = (-90, 0)$ , sufficiently far from the workspace so that it does not interfere with the trajectory.

### C. Expert demonstrations

To generate the expert demonstrations, we design motion planners that build on the motion planning functionality provided by ManiSkill [3]. Concretely, we use the PandaArmMotionPlanningSolver, which internally inherits from BaseMotionPlanningSolver. Given the robot’s current joint configuration and a target end-effector pose, our code calls `move_to_pose_with_screw`, which uses `mplib`’s `plan_screw` function to construct a smooth screw-interpolated path in end-effector space and then computes a corresponding joint-space trajectory that respects the robot’s kinematic limits and avoids self-collisions (and any additional obstacles provided as a point cloud). The resulting sequence of joint positions is then executed in the simulator. On top of this low-level functionality, we design task-specific planners that select appropriate end-effector waypoints (e.g., approach, grasp, lift, detour, place) so that following these screw-based motions solves the manipulation tasks. For our tasks, we build two types of motion planners.

*a) No obstacle:* When there is no obstacle, the planner constructs a simple sequence of end-effector waypoints to pick and place the cube. The planner first searches for a valid grasp pose on the red cube using the cube’s oriented bounding box by evaluating several candidate wrist orientations and selecting the first one reachable by the low-level planner. It then computes a pre-grasp (reach) pose located slightly above this grasp pose. The robot moves to the reach pose, descends to the grasp pose, and closes the gripper to grasp the cube. After grasping, the planner lifts the cube vertically by moving to a pose offset above from the grasp pose. Finally,

it computes a stacking pose directly above the green cube, moves in a straight line to this pose (via a screw-based end-effector interpolation), and opens the gripper to release the red cube on top. For an example of such a demonstration, see [video\\_no\\_obst\\_demo.mp4](#).

*b) With obstacle:* When there is an obstacle between the two cubes, the planner constructs a different path. It still starts by computing a suitable grasp pose on the red cube using its oriented bounding box and selecting a feasible wrist yaw. Once a valid grasp pose is found, the planner first computes a high hover height above the red cube that safely clears the obstacle. The planner then moves the end-effector to the hover pose directly above the grasp pose at this height. From there, it performs a short vertical descent to a reach pose just above the grasp, moves to the grasp pose, and closes the gripper. After grasping, it lifts the cube back up to a lift height. To take a detour around the obstacle, it constructs a three-segment path in the horizontal plane: (1) a lateral “side-step” pose offset perpendicular to the red  $\rightarrow$  green cube direction at lift height, (2) a straight motion toward the target region near the green cube while maintaining that lateral offset (i.e., a straight line parallel to the red  $\rightarrow$  green cube direction), and (3) an “undo side-step” pose that removes the lateral offset so the end-effector is directly above the desired stacking pose. Finally, it descends from this aligned lift pose to the place pose above the green cube and opens the gripper. For an example of such a demonstration, see [video\\_ood\\_obst\\_demo.mp4](#). Note that the trajectories between these poses are executed by successive calls to `move_to_pose_with_screw`, so the low-level planner generates smooth screw-interpolated joint trajectories between each pair of poses, while the obstacle avoidance behavior comes from the construction of the detour in end-effector space. Note also that the choice of which side to use for the detour is randomized, so the demonstrations naturally include trajectories that pass on either side of the obstacle.

### D. Training and evaluation

We use a 1D Conditional UNet architecture with 4.5 million parameters, trained using the MSE noise-prediction objective described in Subsection III-A. Models are optimized with AdamW [14], using weight decay and a cosine learning-rate schedule with 500 warmup steps over the full training horizon.

The models operate in a state-based observation mode. For each training batch, the model receives a sequence of low-dimensional state observations and predicts a sequence of future end-effector actions. Actions are represented as tensors of shape  $(BS, 16, 4)$ , where  $BS$  is the batch size, 16 is the prediction horizon, and each action vector is  $a_t = [\Delta x, \Delta y, \Delta z, g]$ , with  $\Delta x$ ,  $\Delta y$ , and  $\Delta z$  denoting normalized translational displacements of the tool center point (TCP) and  $g$  a normalized gripper command. State observations have shape  $(BS, 2, N_s)$ , where the observation horizon is 2. The state vector of dimension  $N_s$  contains robot proprioception and task-specific object information. The robot contributes 18 dimensions: 9 joint positions (seven arm joints and two finger joints) and 9 joint velocities. Since the Panda arm

is fixed to the table, no floating-base pose is included. The remaining  $N_s - 18$  dimensions encode task-level information. For the stack-cube task without obstacles,  $N_s = 48$ , due to 30 additional degrees of freedom—the poses (position + quaternion;  $3 \times 7 = 21$ ) of the TCP and both cubes, and three relative displacement vectors between objects ( $3 \times 3 = 9$ ). For the obstacle variant, one additional object (the blue block) and a fourth displacement vector increase the state dimension to  $N_s = 58$ .

All models are trained for 100,000 iterations with a batch size of 512 on a single GPU, using the same architecture, hyperparameters, and random seed. The only factor that changes across experiments is the composition of the training dataset. In each OOD study (OOD-positions in Subsection IV-A and OOD-obstacles in Subsection IV-B), we begin by training a baseline model on 200 in-distribution demonstrations and confirm high ID success rates (above 90%). We then evaluate this model in the OOD task, observing that performance drops substantially. Next, we collect 200 OOD demonstrations using our motion planners and incrementally inject subsets of these OOD samples into the training dataset, randomly mixing ID and OOD demonstrations. The model is retrained for each dataset variant, and we track how OOD performance improves as more OOD demonstrations are added.

Additionally, we conduct two forms of evaluation: during training and after training. During training, each model is evaluated every 5,000 iterations on 100 parallel tasks, and we report the mean success rate. Furthermore, the maximum number of allowed actions per episode matches the maximum length of the demonstration trajectories: 150 actions for the OOD position tasks (Subsection IV-A) and 300 for the OOD obstacle tasks (Subsection IV-B), due to the longer detour trajectories. After training, we evaluate each model on 400 tasks using the same random seed and the checkpoint with the best success-at-end performance. Overall, this protocol allows us to directly measure how generalization performance scales as we gradually increase the amount of structured OOD data, in line with our stated design goal.

#### IV. RESULTS

##### A. Generalization to new positions

We start training a base DP model on region A of the table with 200 in-distribution demonstrations. After training, the model reaches a success rate of 94.75% in region A, see Fig. 2 for an example. However, it exhibits extremely poor performance in region B, with a success rate consistently below 10% throughout training (see Fig. 3), and of 3.75% in the post-training evaluation. See Fig. 4 for an example of failure in region B. This motivates studying how generalization to region B improves as we gradually augment the training set with OOD demonstrations. We denote the number of OOD demonstrations by  $N_o$ , and we use  $N_o \in \{0, 1, 2, 5, 10, 20, 30\}$ . We report the learning curves for each  $N_o$  in Fig. 5. We observe a clear positive correlation between  $N_o$  and OOD performance: larger values of  $N_o$  consistently yield higher success rates during training. The evaluation results for the best-performing

checkpoints on 400 OOD tasks are shown in Fig. 6, where performance in region B (OOD-B) corresponds to the red curve. Only 2 OOD demonstrations are enough to increase the success rate from 3.75% to 20.25%, 5 demonstrations to 33.25%, and 10 to 57%. With 30 demonstrations, the performance reaches 94.75%. To verify whether the high performance with  $N_o = 30$  is due solely to the additional OOD demonstrations, we also train a model using only these 30 region B demonstrations. In this case, the success rate remains consistently below 40% (Fig. 3). This shows that the model relies on combining knowledge learned from region A with the new region B demonstrations, rather than succeeding from the OOD data alone. Overall, these results show that about 30 demonstrations (15% of the original training dataset) are enough to efficiently generalize to the new environment, and according to Fig. 6, the increase in performance seems close to linear with the number of OOD demonstrations added to the training dataset.

Furthermore, to assess whether adding OOD data harms ID performance, we also track success rates in the ID domain. As shown in Fig. 6, ID performance remains largely unchanged across the different values of  $N_o$ . Moreover, all models exhibit nearly identical ID performance trajectories during training, regardless of the value of  $N_o$  (Fig. 7). This indicates that incorporating OOD demonstrations does not degrade ID behavior.

In addition to evaluating generalization to region B, we study two other OOD settings involving new object positions: OOD-C (region C) and OOD-AB. In the OOD-AB task, the red cube is placed in region A and the green cube in region B. The robot must therefore pick the red cube in region A and stack it on the green cube in region B, a trajectory that is unseen during training—unlike OOD-B and OOD-C, which involve motions that are more locally similar to those in the training data. Figure 6 reports the performance of the models on both tasks.

As shown in the figure, performance in OOD-C is already high with only the 200 region A demonstrations (75.75% success rate), likely because this region is spatially closer to region A and therefore easier to generalize to. Adding demonstrations from region B yields consistent improvements in region C, reaching a 93.25% success rate when  $N_o = 30$ . This suggests that the model may be “interpolating” between trajectories in regions A and B, enabling strong performance in the intermediate region even without region C-specific demonstrations.

For the OOD-AB task, we also observe a steady improvement as  $N_o$  increases, with success rising from 7.75% at  $N_o = 0$  to 32.25% at  $N_o = 30$ . This improvement is plausibly due to the OOD-B demonstrations enhancing the robot’s stacking behavior in region B, partially transferring to the OOD-AB setting. Most failures occur because the robot drops the red cube short of the green cube, placing it on the table instead; see Fig. 8 for an example. Note that if the maximum number of allowed actions per episode (150) was increased, the robot could reattempt the stack and potentially achieve higher success rates. However, under our definition

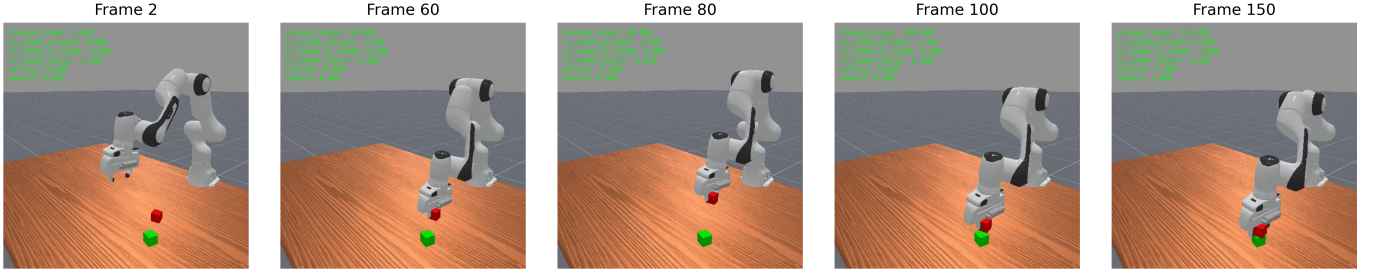


Fig. 2. Example of successful task completion in region A after training DP with 200 demonstrations from region A. Full video in [video\\_end\\_train.mp4](#).

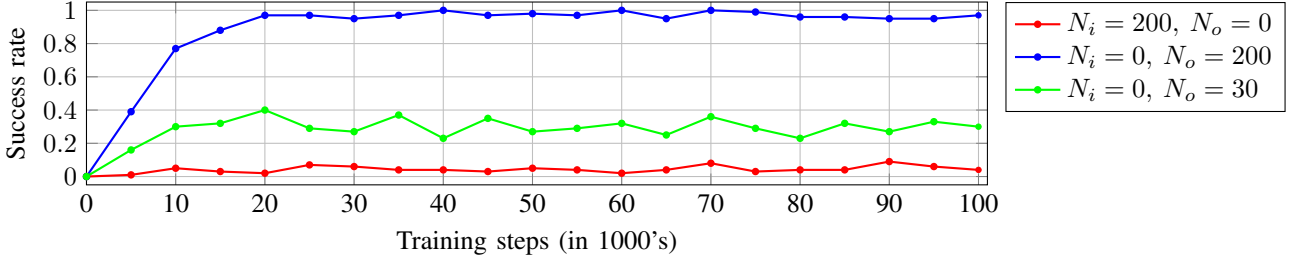


Fig. 3. Success rates throughout training in the OOD-position environment (region B) for the baseline models. Here,  $N_i$  and  $N_o$  denote the numbers of ID and OOD demonstrations in the training dataset, respectively.

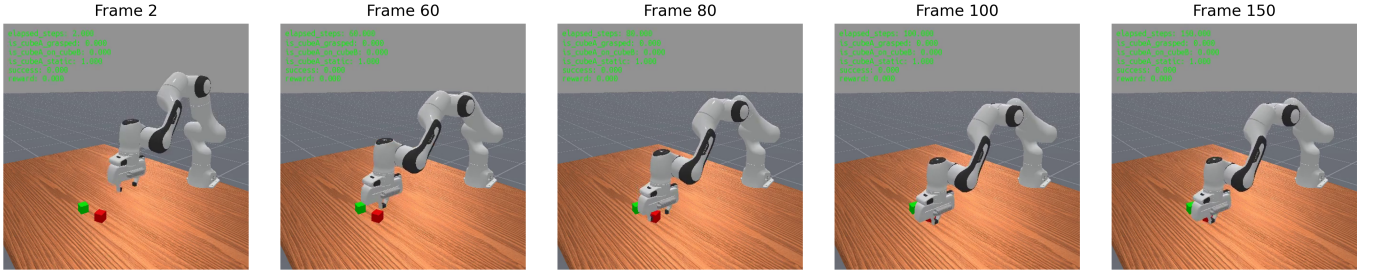


Fig. 4. Example of failure in region B after training DP with 200 demonstrations from region A. Full video in [video\\_ood\\_pos\\_fail.mp4](#).

of success—completing the task within 150 actions—such delayed recoveries count as failures.

### B. Generalization to the presence of obstacles

Analogously to the OOD-position experiments, we first train a baseline DP model on the 200 in-distribution demonstrations (without an obstacle). After training, the model reaches a 98.75% success rate on the stack-cube task without obstacle, but it completely fails in the OOD-obstacle setting, achieving 0% success (see Fig. 9). We observe two failure modes: (1) the robot fails to grasp the red cube, likely because it does not know how to adapt to the altered conditional observation (Fig. 10); and (2) the robot grasps and lifts the red cube but then attempts a straight-line motion toward the green cube, colliding with the block and dropping the cube (Fig. 11).

Using the motion planner designed for the OOD-obstacle task, we collect 200 demonstrations with the obstacle. As a sanity check, we train DP only on these 200 OOD demonstrations, achieving 96% success rate after training (Fig. 9). However, this OOD-trained model completely fails in the no-

obstacle setting, achieving 0% success at the end of training. Thus, generalization from obstacle to no-obstacle is also poor.

Next, we retrain the DP model using the base 200 no-obstacle demonstrations together with varying numbers  $N_o$  of OOD demonstrations, with  $N_o \in \{0, 1, 2, 5, 10, 20, 30, 40, 50, 100\}$ . Evaluations throughout training and post-training evaluation curves are shown in Fig. 12 and Fig. 13, respectively. In general, OOD performance increases with  $N_o$  similarly to the OOD position experiments. Interestingly, a single OOD demonstration boosts the success rate up to 36% during training and 18.25% in the post-training evaluation. However, using two OOD demonstrations produces worse performance: only 7% success in post-training evaluation. We hypothesize that the two demonstrations, performing different trajectories, “interfere” with each other. In fact, we performed analogous experiments with other individual demonstrations and pairs of demonstrations and consistently obtained a reduced performance of two OOD demonstrations compared to a

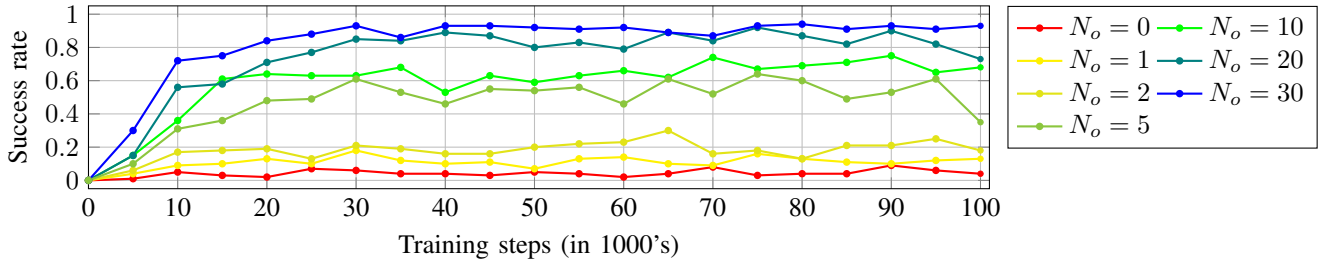


Fig. 5. OOD evaluations (in region B) of the models throughout training, for different numbers  $N_o$  of OOD demonstrations in region B.

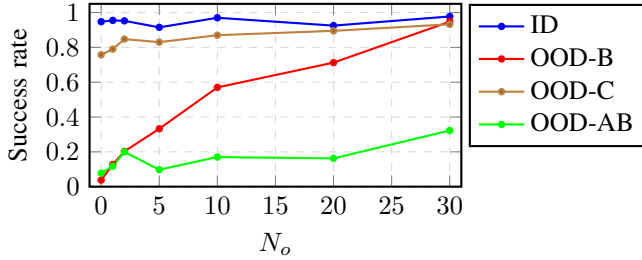


Fig. 6. Success rate vs. the number of OOD demonstrations mixed in the dataset,  $N_o$ , for the OOD position tasks.

single demonstration. For larger values of  $N_o$ , performance increases approximately linearly up to  $N_o = 40$ , reaching 80.25%. Beyond that point, improvements slow down, reaching 89.75% at  $N_o = 100$ . To assess whether the OOD performance comes solely from the OOD demonstrations, we train DP using only 40 OOD demonstrations (Fig. 9). In this case, success rate remains mostly below 60% during training and reaches only 53% in the post-training evaluation, compared to 81.25% when using the mixed dataset with  $N_i = 200$  and  $N_o = 40$ . Hence, the OOD generalization comes from combining the OOD and ID demonstrations.

We also track the evolution of performance in the original no-obstacle environment. As shown in Fig. 13 (blue curve), ID performance remains nearly constant across  $N_o$ , with only a slight drop at  $N_o = 100$  (from 97% for  $N_o = 50$  to 92% for  $N_o = 100$ ). Training curves in Fig. 14 also show similar trajectories across all models, with the  $N_o = 100$  model lagging slightly behind, likely due to the reduced frequency of ID examples early in training.

It is also important to note that the models learn to compose both modes of behavior. After training, DP reliably determines whether the obstacle lies between the cubes. If the path is unobstructed, it executes a direct straight-line trajectory; otherwise, it performs the correct detour. See Fig. 15 for an example of successful obstacle avoidance and Fig. 16 for a successful ID execution. This multimodal capability emerges already at  $N_o = 1$  and becomes increasingly accurate as  $N_o$  grows. We also observe emergent behaviors not present in the training data. In one case, the model produces a trajectory composed of four linear segments, whereas all demonstrations contain only one or three segments; see Fig. 17. In another

case, the robot completes the task using only two linear motions, achieving a faster solution despite never having seen such a strategy during training; see Fig. 18.

Finally, we investigate the effect of the ratio between ID and OOD demonstrations. In the preceding experiments we used  $N_i = 200$ , meaning that 10 OOD demonstrations constitute only about 5% of the dataset. This raises the question: would performance change if fewer ID demonstrations were used (e.g.,  $N_i = 100$  or  $N_i = 50$ )? To address this, we repeat the experiments with  $N_i \in \{50, 100, 200\}$  and  $N_o \in \{1, 5, 10, 20, 40, 100\}$ . The evaluation results for all combinations are shown in Fig. 19. Interestingly, for all three values of  $N_i$ , we observe an inflection point at  $N_o = 40$ , beyond which improvements in OOD performance begin to slow down. We also observe a consistent ordering in ID performance across  $N_i$ : the dotted curves show that models trained with more ID data perform better— $N_i = 200$  achieves the highest success rates, followed by  $N_i = 100$ , and then  $N_i = 50$ . The solid curves (OOD evaluations) follow a similar ordering for  $N_o \in \{20, 40\}$ :  $N_i = 200$  performs best, followed by  $N_i = 100$ , and finally  $N_i = 50$ . However, for  $N_o \geq 40$ , the curves begin to converge, indicating that sufficient OOD data can compensate for reduced ID coverage. Importantly, the results do not suggest that having more ID data harms OOD generalization; rather, the models appear able to integrate both modes and perform effectively in both domains. Moreover, when  $N_i = 50$ , ID performance substantially improves as  $N_o$  increases—from 42% to 75%—reinforcing the idea that the model leverages information learned from both modes to enhance its behavior in both ID and OOD tasks.

## V. DISCUSSION AND FUTURE WORK

Our results reveal that diffusion policies exhibit strong data-driven generalization under structured distribution shifts. In many cases, only a small number of OOD demonstrations (about 10–15% of the original dataset) are sufficient for the policy to adapt to a new environment, boosting performance from below 5% to over 90% success on OOD-position tasks, and from 0% to over 70% on OOD-obstacle tasks. Moreover, success rates scale close to linearly with the number of OOD demonstrations in both settings for small values of  $N_o$  (up to roughly 30 for OOD-positions and 40 for OOD-obstacles). However, OOD-obstacle generalization requires substantially more OOD data to reach 90% success (over 100 demonstra-

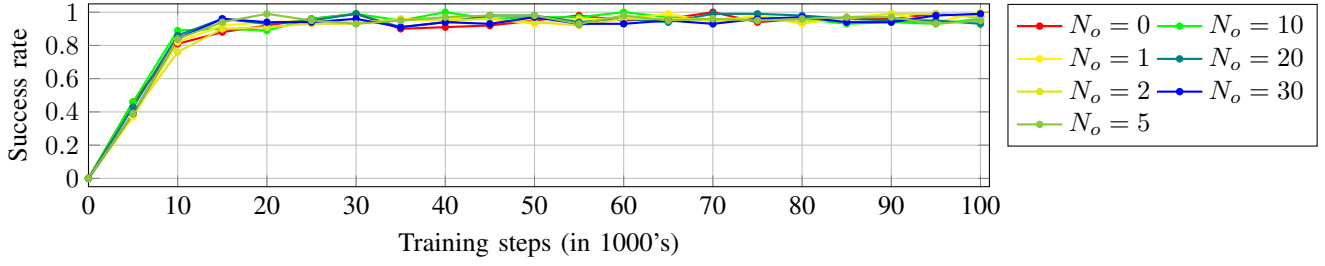


Fig. 7. ID evaluations of the models throughout training, for different numbers  $N_o$  of OOD demonstrations in region B.

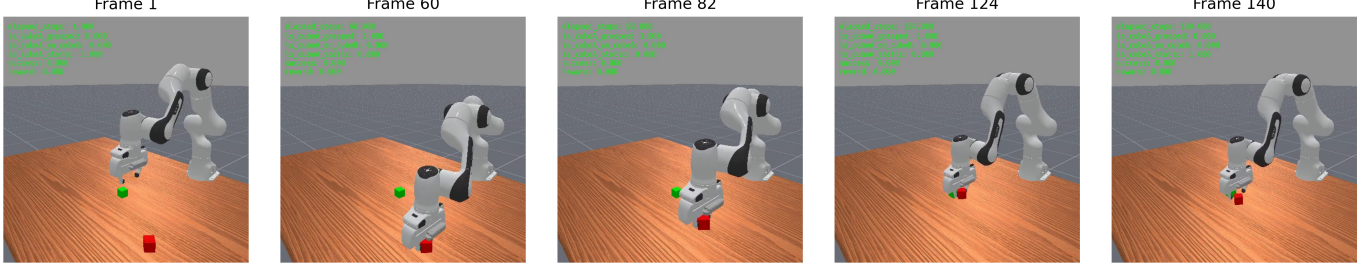


Fig. 8. Example of failure in the OOD-AB task after training DP with 200 demonstrations from region A. Full video in [video\\_ood\\_pos3\\_fail.mp4](#).

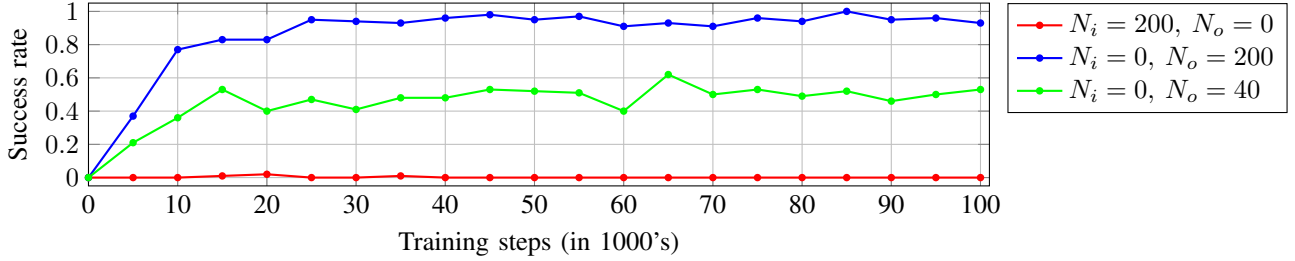


Fig. 9. Success rates throughout training in the OOD obstacle environment for the baseline models.

tions, compared to only 30 for OOD-position), likely reflecting the increased complexity of the obstacle task. On the other hand, adding OOD data does not degrade ID performance; in some cases, OOD demonstrations even improve ID behavior, suggesting that the model benefits from jointly learning multiple modes of behavior.

Several interesting phenomena emerge from our experiments. First, we find evidence of interpolation-based generalization, where combining demonstrations from distinct spatial regions enables strong performance in intermediate regions not seen during training. Second, in the OOD-obstacle experiments we observe that when only two OOD demonstrations are provided, performance is consistently worse than with a single demonstration, hinting at possible “interference” effects between pairs of incompatible trajectories. Finally, we see evidence that the models can generate trajectories unseen during training, such as four-segment and two-segment detours around the obstacle, whereas demonstrations only contain three-segment trajectories. These effects raise questions about how diffusion policies internally organize and compose behavior, and what representations they learn to support adaptation

to new domains.

There are several promising directions for future work. First, our experiments rely on state-based observations; extending the study to vision-based diffusion policies would reveal how visual factors (e.g., lighting, color, distractors) influence generalization under OOD shifts. Although we conducted preliminary experiments with vision-based observations, we found that DP required many more iterations to learn, making such experiments substantially more time-consuming, so we focused on state-based observations here. Second, a more systematic investigation of interpolation and extrapolation effects could help clarify the mechanisms by which diffusion models acquire and integrate behaviors across different tasks and settings. Third, exploring a wider range of object types and manipulation tasks—including more complex and longer-horizon scenarios, both in simulation and in the real world—would help clarify the limits of diffusion-policy generalization. Finally, it would be valuable to study compositional generalization strategies; for instance, whether a single monolithic model trained on datasets A and B outperforms a composition of specialized models trained separately on A and B.

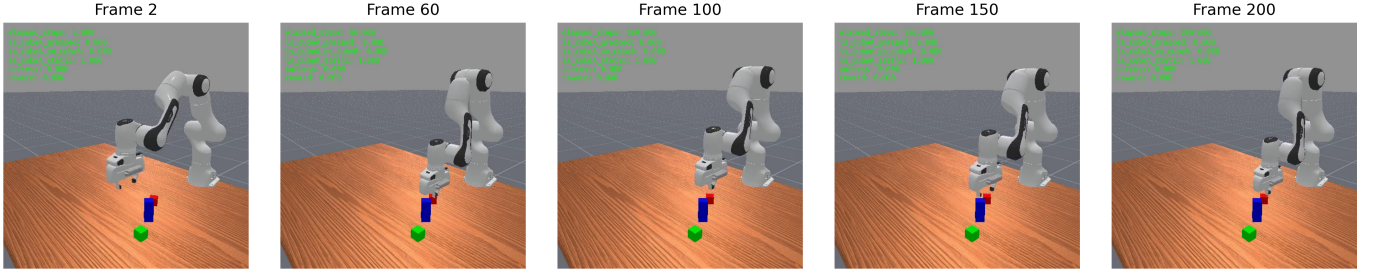


Fig. 10. Example of failure (type 1) in presence of an obstacle, after training the DP model with 200 demonstrations without obstacle. Full video in [video\\_ood\\_obst\\_fail\\_type1.mp4](#).

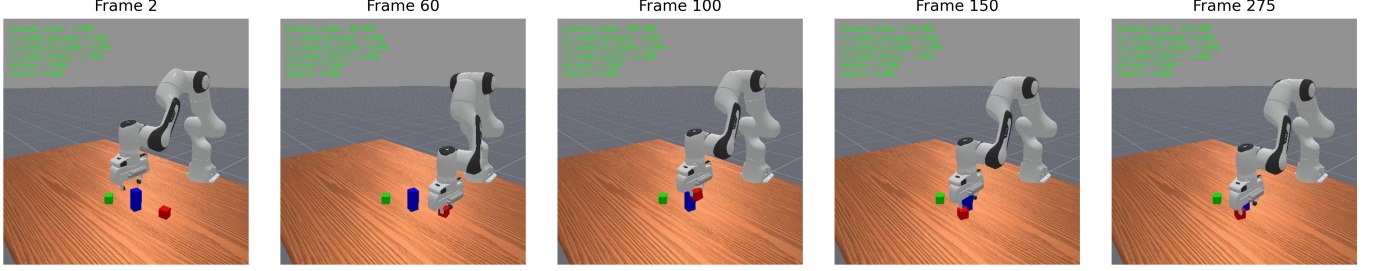


Fig. 11. Example of failure (type 2) in presence of an obstacle, after training the DP model with 200 demonstrations without obstacle. Full video in [video\\_ood\\_obst\\_fail\\_type2\\_a.mp4](#).

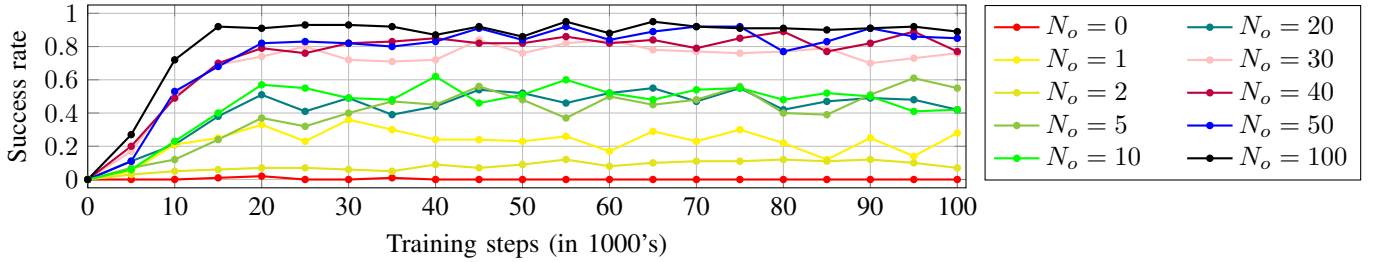


Fig. 12. OOD evaluations (with an obstacle) of the models throughout training, for different numbers  $N_o$  of OOD demonstrations with an obstacle.

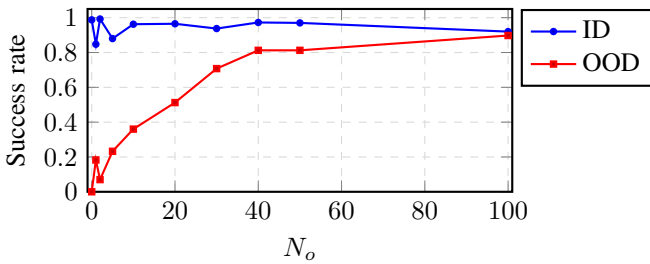


Fig. 13. Success rate vs. the number of OOD demonstrations mixed in the dataset,  $N_o$ , for the OOD obstacle tasks.

## VI. CONCLUSIONS

We have presented a systematic study of out-of-distribution generalization in diffusion policies across two types of distribution shifts: novel object positions and the presence of obstacles. Our experiments show that diffusion policies can be rapidly adapted to new domains with only small amounts

of OOD data, boosting OOD success rates from below 5% to above 70% after adding as few as 20–30 additional demonstrations in both OOD-position and OOD-obstacle settings. Additionally, we find that success rates scale close to linearly in both tasks for  $N_o$  up to roughly 30–40. Overall, our findings suggest that diffusion policies are highly adaptable and capable of composing multiple behavioral modes when exposed to heterogeneous demonstrations, enabling generalization to under-represented tasks. At the same time, OOD-obstacle generalization requires substantially more OOD data to reach 90% success compared to the OOD-position task, indicating that the data efficiency of generalization depends on task complexity: more structured or multimodal tasks may require larger amounts of OOD data before strong improvements emerge. Extending this study to longer-horizon tasks, more complex manipulation scenarios, and vision-based observation modes—both in simulation and in the real world—will be essential for developing a deeper understanding of diffusion-policy generalization in practical robotics settings.

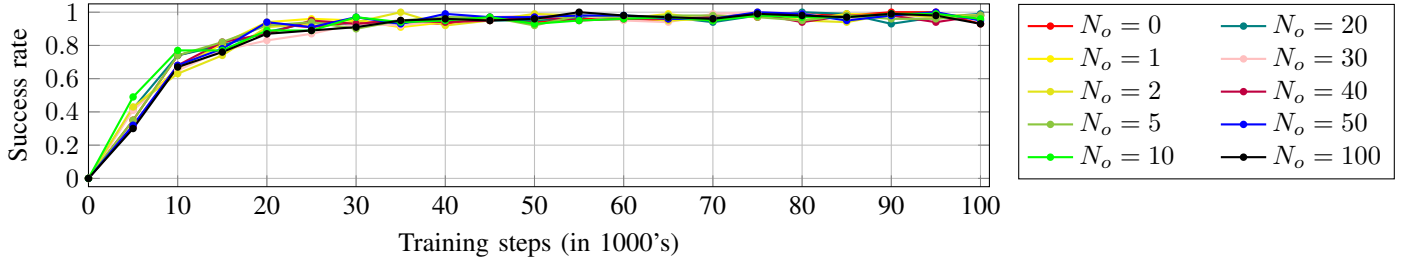


Fig. 14. ID evaluations of the models throughout training, for different numbers  $N_o$  of OOD demonstrations with an obstacle.

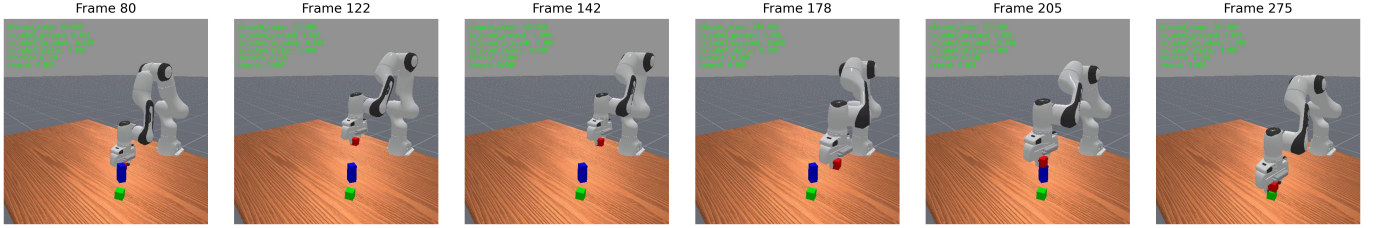


Fig. 15. Example of successful task completion in the presence of an obstacle, after training the DP model with  $N_i = 200$  and  $N_o = 40$ . Full video in [video\\_ood\\_obst\\_success.mp4](#).

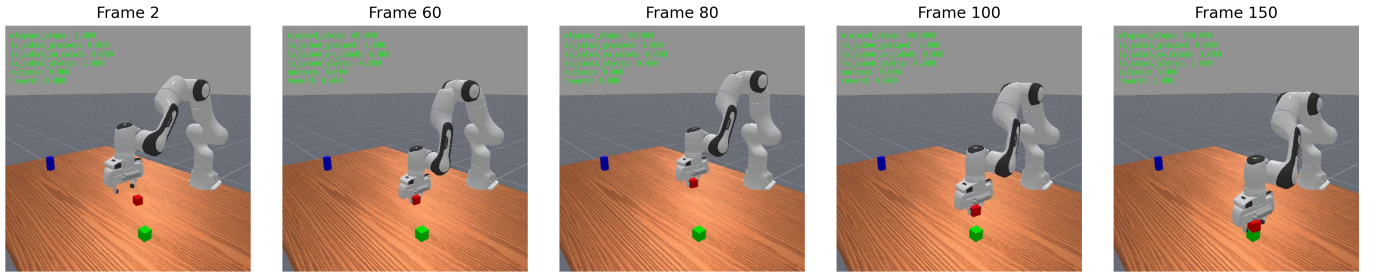


Fig. 16. Example of successful task completion with no obstacle, after training the DP model with  $N_i = 200$  and  $N_o = 40$ . Full video in [video\\_ood\\_obst\\_id\\_success.mp4](#).

#### CONTRIBUTIONS MADE BY EACH TEAM MEMBER

a) *Jack Benarroch Jedlicki*: contributed to the creation of the training and evaluation environments, the design of the motion planners, training of the models, and the writing of the report and video presentation.

b) *Kushal Chattopadhyay*: contributed to the evaluation of the models, analysis and reporting of the results, and the writing of the report and video presentation.

#### NOTE ON GENERATIVE AI USE

During this project, we used large language models (GPT-5 and GPT-5.1) to provide writing suggestions and to help improve code snippets.

#### ACKNOWLEDGMENT

We thank the authors of the ManiSkill and SAPIEN frameworks [3, 12, 13] and of the Diffusion Policy framework [5] for making their work publicly available. We also thank the FASRC cluster from Harvard University for providing the computational resources used to conduct our experiments.

#### REFERENCES

- [1] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.
- [2] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.
- [3] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.

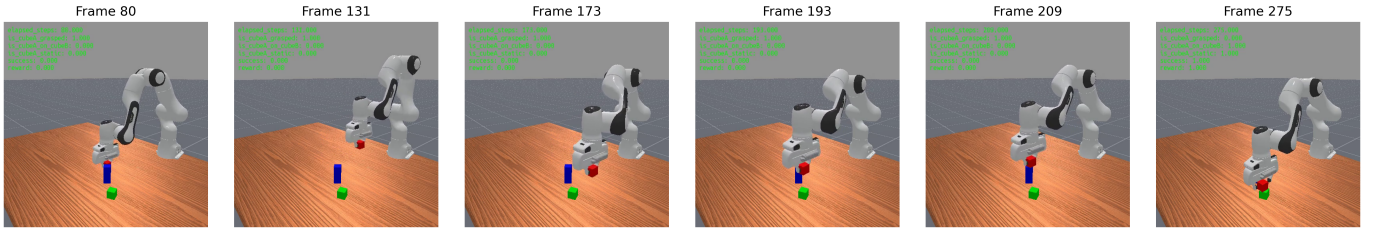


Fig. 17. Example of new behavior 1: the robot learns to generate a 4-segment path in the presence of an obstacle, unseen during training. Full video in [video\\_ood\\_obst\\_4\\_trajs.mp4](#).

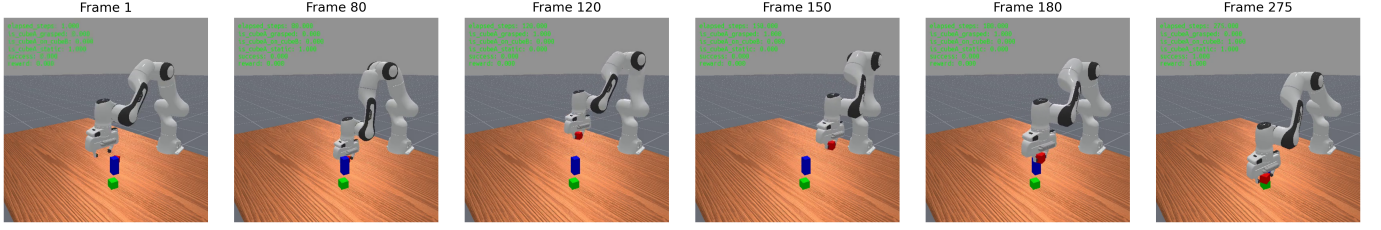


Fig. 18. Example of new behavior 2: the robot learns to generate a 2-segment path in the presence of an obstacle, unseen during training. Full video in [video\\_ood\\_obst\\_2\\_trajs.mp4](#).

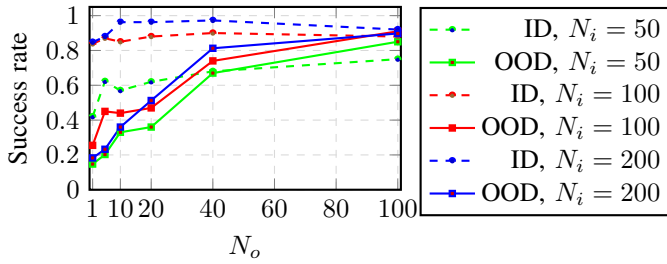


Fig. 19. Success rate vs. the number of OOD demonstrations mixed in the dataset,  $N_o$ , for the OOD position tasks, for different numbers of ID demonstrations  $N_i$ .

- [4] Jensen Gao, Suneel Belkhale, Sudeep Dasari, Ashwin Balakrishna, Dhruv Shah, and Dorsa Sadigh. A taxonomy for evaluating generalist robot policies, 2025. URL <https://arxiv.org/abs/2503.01238>.
- [5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [6] Yilun Du, Yoonho Li, Ashvin Singh, Jeong Joon Park, Joshua B Tenenbaum, and Pulkit Agrawal. 3d diffusion policy for robot control. *arXiv preprint arXiv:2306.17399*, 2023.
- [7] Nolan Wagener, Peter Wirsberger, Ayush Jain, Mingqi Sun, Tim Rocktäschel, and Edward Grefenstette. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2303.04156*, 2023.
- [8] Tim Pearce, Shengjie Luo, Alex Lee, Chelsea Finn, and Sergey Levine. Imitate, then reward: Two-stage diffusion

- policy learning. *arXiv preprint arXiv:2307.05679*, 2023.
- [9] Xinyue Wang, Edward Grefenstette, and Tim Rocktäschel. Diffusion policies as an expressive policy class for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022.
- [10] Xinyue Wang, Dineshkrishna Misra, Yixin Chen, Edward Grefenstette, and Tim Rocktäschel. Diffusion q-learning: Offline rl with diffusion models. *arXiv preprint arXiv:2303.08748*, 2023.
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [12] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [13] Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. PartNet: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.